

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия,
Системное и прикладное программное обеспечение

Дисциплина: Основы программной инженерии

Лабораторная работа №2
Вариант №160488

Выполнил:
Карнажицкий Максим Романович
Группа: Р3211

Проверил:
Осипов С. В.

г. Санкт-Петербург, 2026

Содержание

1. Описание задания	2
2. Программная реализация	3
2.1. Git	3
2.2. SVN	6
3. Вывод	11

1. Описание задания

Сконфигурировать в своём домашнем каталоге репозитории svn и git и загрузить в них начальную ревизию файлов с исходными кодами (в соответствии с выданным вариантом).

Воспроизвести последовательность команд для систем контроля версий svn и git, осуществляющих операции над исходным кодом, приведённые на блок-схеме.

При составлении последовательности команд необходимо учитывать следующие условия:

- Цвет элементов схемы указывает на пользователя, совершившего действие (красный - первый, синий - второй).
- Цифры над узлами - номер ревизии. Ревизии создаются последовательно.
- Необходимо разрешать конфликты между версиями, если они возникают.

Вариант

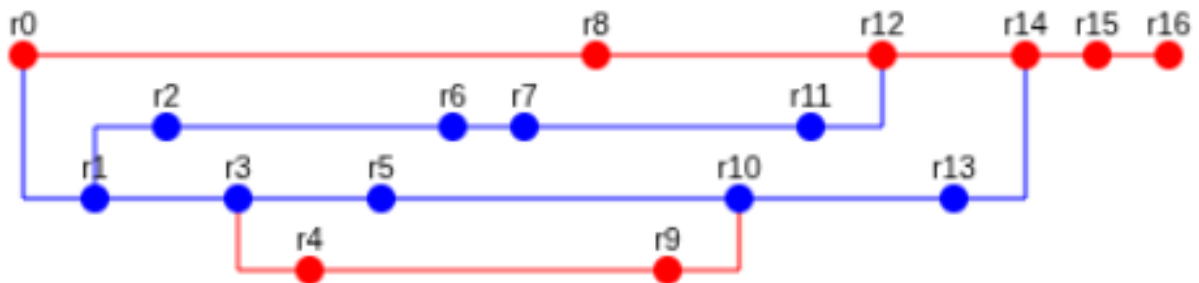


Рис. 1. Задание варианта 160488

2. Программная реализация

Перед реализацией скриптов для `git` и `svn` я создал следующую структуру директорий:

```
.
├── commits
│   ├── commit0.zip
│   ├── ...
│   ├── commit16.zip
│   └── unzip_all.sh
├── git.sh
└── svn.sh
```

Для удобной работы сначала разархивируем полученные ревизии в ту же директорию `commits/` с помощью такого скрипта (`commits/unzip_all.sh`):

```
for f in commit*.zip; do unzip -d "${f%.zip}" "$f"; done
```

Сами реализации содержатся в `git.sh`, `svn.sh`

2.1. Git

Для удобной работы я написал процедуры, чтобы не повторять много раз один и тот же код.

```
#!/bin/bash

init() {
    rm -rf ./git
    mkdir git
    cd git
    git init
}

set_red() {
    git config --local user.name red
    git config --local user.email red@example.com
}

set_blue() {
    git config --local user.name blue
    git config --local user.email blue@example.com
}

set_branch() {
    # $1          : branch_number
    # $2 <opt>    : -b if new branch
    if [ "$1" == "master" ]; then
        git checkout $2 master
    else
        git checkout $2 b$1
    fi
}

commit() {
    # arg 1 : commit_number (integer >= 0)
    rm -rf *
```

```

    cp ../commits/commit$1/* .
    git add .
    git commit -m "[feat] commit number $1"
}

merge() {
    # arg 1 : branch_number
    git merge b$1 --no-commit
}

# go

init

# r0
set_red
commit 0

# r1
set_blue
set_branch 3 -b
commit 1

# r2
set_branch 2 -b
commit 2

# r3
set_branch 3
commit 3

# r4
set_red
set_branch 4 -b
commit 4

# r5
set_blue
set_branch 3
commit 5

# r6
set_branch 2
commit 6

# r7
commit 7

# r8
set_red
set_branch master
commit 8

# r9
set_branch 4
commit 9

# r10
set_blue
set_branch 3
merge 4
commit 10

# r11
set_branch 2
commit 11

# r12
set_red
set_branch master

```

```

merge 2
commit 12

# r13
set_blue
set_branch 3
commit 13

# r14
set_red
set_branch master
merge 3
commit 14

# r15
commit 15

# r16
commit 16

```

```

$ cd git
$ git log --graph --oneline

```

```

* 1b3037f (HEAD -> master) [feat] commit number 16
* 473c337 [feat] commit number 15
* 7faecca [feat] commit number 14
|\
| * cf5662e (b3) [feat] commit number 13
| * fbe5ca6 [feat] commit number 10
| |\
| | * aa6a6a7 (b4) [feat] commit number 9
| | * 2bf01dd [feat] commit number 4
| | * | c3f66b7 [feat] commit number 5
| | /
| * e78f06b [feat] commit number 3
* | 4a12af6 [feat] commit number 12
|\ \
| * | 356674d (b2) [feat] commit number 11
| * | 53331ef [feat] commit number 7
| * | abd2569 [feat] commit number 6
| * | 5e08af6 [feat] commit number 2
| | /
| * 1d4cf85 [feat] commit number 1
* | 8b780bc [feat] commit number 8
| /
* e8bf0f3 [feat] commit number 0

```

История коммитов отображается нелинейно, но соответствует блок-схеме.

2.2. SVN

```
#!/bin/bash

init() {
    rm -rf ./svn
    mkdir svn
    cd svn

    export SVN_REMOTE_URL="file://$(pwd -P)/svn-repo"
    export SVN_COMMITS="$(pwd -P)/../commits"
    export SVN_CURRENT_USER=red

    svnadmin create svn-repo
    svn mkdir $SVN_REMOTE_URL/trunk $SVN_REMOTE_URL/branches -m "basic file structure is initied" --
username=$SVN_CURRENT_USER
}

set_red() {
    export SVN_CURRENT_USER=red
}

set_blue() {
    export SVN_CURRENT_USER=blue
}

branch() {
    # $1      : source_branch_number (or "trunk")
    # $2      : target_branch_number

    if [ "$1" = "trunk" ]; then
        svn copy $SVN_REMOTE_URL/trunk $SVN_REMOTE_URL/branches/b$2 \
-m "create branch $2 from trunk" --username=$SVN_CURRENT_USER
    else
        svn copy $SVN_REMOTE_URL/branches/b$1 $SVN_REMOTE_URL/branches/b$2 \
-m "create branch $2 from $1" --username=$SVN_CURRENT_USER
    fi
}

switch() {
    # $1      : target_branch_number (or "trunk")

    if [ "$1" = "trunk" ]; then
        svn switch $SVN_REMOTE_URL/trunk
    else
        svn switch $SVN_REMOTE_URL/branches/b$1
    fi
}

merge() {
    # $1      : tomerge_branch_number (no trunk no)

    svn merge $SVN_REMOTE_URL/branches/b$1 --non-interactive
}

commit() {
    # $1      : commit_number

    svn rm * --force
    cp $SVN_COMMITS/commit$1/* .
    svn add * --force

    svn commit -m "[feat] commit number $1" --username=$SVN_CURRENT_USER
}

create_working_copy() {
    svn checkout $SVN_REMOTE_URL/trunk working_copy
    cd working_copy
}
```

```

}

# go

init
create_working_copy

# r0
set_red
commit 0

# r1
set_blue
branch trunk 3
switch 3
commit 1

# r2
branch 3 2
switch 2
commit 2

# r3
switch 3
commit 3

# r4
set_red
branch 3 4
switch 4
commit 4

# r5
set_blue
switch 3
commit 5

# r6
switch 2
commit 6

# r7
commit 7

# r8
set_red
switch trunk
commit 8

# r9
switch 4
commit 9

# r10
set_blue
switch 3
merge 4
commit 10

# r11
switch 2
commit 11

# r12
set_red
switch trunk
merge 2
commit 12

# r13

```



```

set_blue
switch 3
commit 13

# r14
set_red
switch trunk
merge 3
# тут возникает конфликт, который если не решить, упадет core dump :)))
svn resolve --accept working WHXsjyye7f.YDN
commit 14

# r15
commit 15

# r16
commit 16

```

Для проверки корректности выведем список ревизий для каждой ветки **trunk**

```

$ cd svn/working_copy
$ svn log ^/trunk

```

```

-----
r21 | red | 2026-04-15 20:22:03 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 16
-----
r20 | red | 2026-04-15 20:22:03 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 15
-----
r19 | red | 2026-04-15 20:22:03 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 14
-----
r17 | red | 2026-04-15 20:22:01 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 12
-----
r13 | red | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 8
-----
r2 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 0
-----
r1 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
basic file structure is initied
-----

```

b2

```

$ svn log ^/branches/b2

```

```

-----
r16 | blue | 2026-04-15 20:22:01 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 11

```

```
-----
r12 | blue | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 7
```

```
-----
r11 | blue | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 6
```

```
-----
r6 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 2
```

```
-----
r5 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
create branch 2 from 3
```

```
-----
r4 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 1
```

```
-----
r3 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
create branch 3 from trunk
```

```
-----
r2 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 0
```

```
-----
r1 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
basic file structure is initied
-----
```

b3

```
$ svn log ^/branches/b3
```

```
-----
r18 | blue | 2026-04-15 20:22:01 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 13
```

```
-----
r15 | blue | 2026-04-15 20:22:01 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 10
```

```
-----
r10 | blue | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 5
```

```
-----
r7 | blue | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 3
```

```
-----
r4 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 1
```

```
-----
r3 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
create branch 3 from trunk
```

```
-----
r2 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
```

```
[feat] commit number 0
-----
```

```
r1 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
basic file structure is initied
-----
```

b4

```
$ svn log ^/branches/b4
```

```
-----
r14 | red | 2026-04-15 20:22:01 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 9
-----
r9 | red | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 4
-----
r8 | red | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
create branch 4 from 3
-----
r7 | blue | 2026-04-15 20:22:00 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 3
-----
r4 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 1
-----
r3 | blue | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
create branch 3 from trunk
-----
r2 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
[feat] commit number 0
-----
r1 | red | 2026-04-15 20:21:59 +0300 (Wed, 15 Apr 2026) | 1 line
basic file structure is initied
-----
```

Видно, что история в целом совпадает.

3. Вывод

В ходе лабораторной работы я улучшил навыки работы с системой контроля версий `Git`, а также впервые поработал с системой `Subversion`. Я инициализировал репозитории `SVN` и `Git` в домашнем репозитории, написал скрипты для имитации совместной работы над репозиториями с несколькими ветками в соответствии с блок-схемой своего варианта. Изучил несколько способов разрешения конфликтов.